



High Performance Computing

Dr.Bheemappa H
IIIT, Sricity
bheemappa.h@iiit.in

M1:

32

- Motivating Parallelism, Scope of Parallel Computing,
- Introduction to HPC:
- Parallel Programming Platforms
 - Implicit Parallelism: Trends in Microprocessor Architectures
 - Limitations of Memory System Performance
- Dichotomy of Parallel Computing Platforms
 - Control Structure of Parallel Platforms
 - Communication Model of Parallel Platforms
- Physical Organization of Parallel Platforms
 - Architecture of an Ideal Parallel Computer
 - Interconnection Networks for Parallel Computers

Explicitly Parallel Platforms

33

- Dichotomy of Parallel Computing Platforms
 - An **explicitly parallel program** must specify **concurrency and interaction between concurrent subtasks**.
 - Control structure
 - Parallelism can be expressed at various levels of granularity - from instruction level to processes
- Control Structure of Parallel Programs
 - Processors either operate under the centralized control of a single control unit or work independently.
 - **Single** control unit that dispatches the same instruction to various processors (that work on different data)

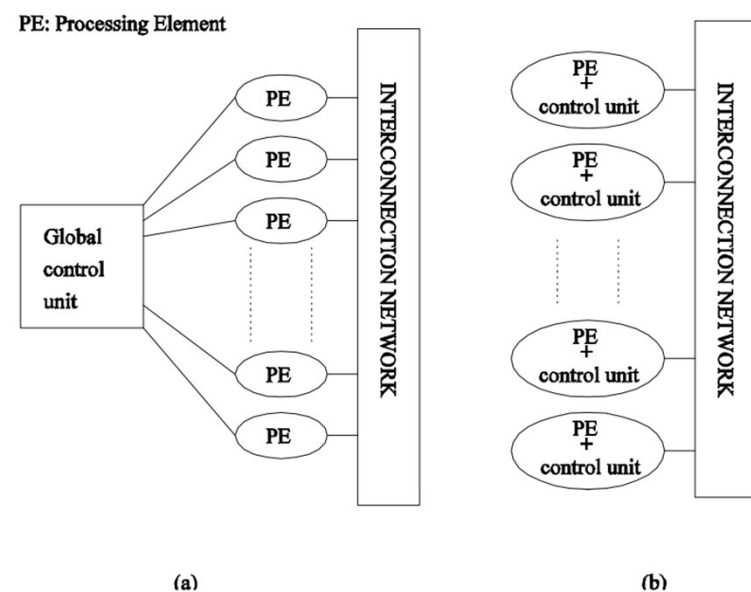
Parallelism

34

- Classes of parallelism in applications:
 - Data-Level Parallelism (DLP)
 - Task-Level Parallelism (TLP)
- Classes of architectural parallelism:
 - Instruction-Level Parallelism (ILP)
 - Vector architectures/Graphic Processor Units (GPUs)
 - Thread-Level Parallelism
 - Request-Level Parallelism

Control Structure of Parallel Programs

- Single instruction stream, multiple data stream (SIMD)
 - If there is a single control unit that dispatches the same instruction to various processors
- multiple instruction stream, multiple data stream (MiMD)
 - If each processor has its own control unit, each processor can execute different instructions on different data items



Parallelism from single instruction on multiple processors(SIMD)

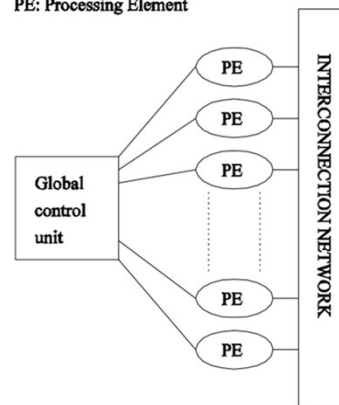
36

- a single control unit dispatches instructions to each processing unit.
- same instruction is executed synchronously by all processing units.
- MMX units in Intel processors and DSP chips
- Structured computations on parallel data structures such as **arrays**, often is necessary to **selectively turn off operations on certain data items**.
- Conditional execution can be detrimental to the performance of SIMD processors and therefore must be used with care.

• Example

- add instruction is dispatched to all processors and executed concurrently by them.

PE: Processing Element



(a)

```
for (i = 0; i < 1000; i++)
    c[i] = a[i] + b[i];

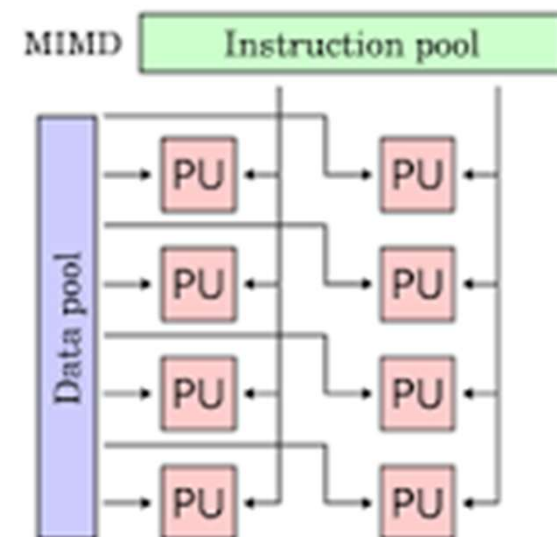
c[0] = a[0] + b[0]; c[1] = a[1] + b[1];
```

<https://bheemhh.github.io/#/HPC26>

Multiple Instruction stream, Multiple Data stream (MIMD)

37

- each processing element is capable of executing a different program independent of the other processing elements
- Each of the multiple programs can be inserted into one large if-else block with conditions specified by the task identifiers.
 - A simple variant of this model, called the single program multiple data (SPMD) model.
 - An example of MIMD system is **Intel Xeon Phi**,



- SIMD computers require less hardware than MIMD computers because they have only one global control unit.
- SIMD computers require less memory because only one copy of the program needs to be stored.
- While MIMD is complex in terms of complexity than SIMD.
- MIMD is more efficient in terms of performance than SIMD.

M1:

39

- Motivating Parallelism, Scope of Parallel Computing,
- Introduction to HPC:
- Parallel Programming Platforms
 - Implicit Parallelism: Trends in Microprocessor Architectures
 - Limitations of Memory System Performance
- Dichotomy of Parallel Computing Platforms
 - Control Structure of Parallel Platforms
 - Communication Model of Parallel Platforms
- Physical Organization of Parallel Platforms
 - Architecture of an Ideal Parallel Computer
 - Interconnection Networks for Parallel Computers

Communication Model of Parallel Platforms

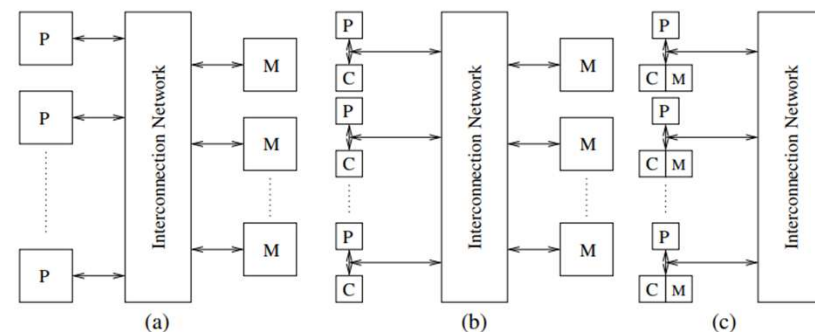
40

- Data exchange between parallel tasks
 - accessing a shared data space (shared address-space machines or **multiprocessors**)
 - Exchanging messages(message passing platforms or **multicomputers**).
- Shared-Address-Space Platforms
 - common data space that is accessible to all processors.
 - Processors interact by modifying data objects stored in this shared-address-space.
 - If the time taken by a processor to access any memory word in the system global or local is identical, the platform is classified as a uniform memory access (UMA), else, a non uniform memory access (NUMA) machine

NUMA and UMA Shared-Address-Space Platforms

41

- NUMA and UMA architectures are defined only in terms of **memory access times** and not cache access times.
- Sun Ultra HPC servers-class of NUMA multiprocessors.



Typical shared-address-space architectures: (a) Uniform-memory-access shared-address-space computer; (b) Uniform-memory-access shared-address-space computer with caches and memories; (c) Non-uniform-memory-access shared-address-space computer with local memory only.

NUMA and UMA Shared-Address-Space Platforms

42

Presence Global Memory (RAM) :

- presence of a global memory space makes (parallel) programming such platforms much easier.
- operations require mutual exclusion for concurrent accesses.
- Read/write -interactions are, however, harder to program than the read-only interactions,

Presence of caches on processors

- raises the issue of multiple copies of a single memory word being manipulated by two or more processors at the same time.
 - Ensuring that **concurrent operations** on multiple copies of the same memory word have well-defined semantics (**cache coherence**)
 - address translation mechanism that locates a memory word in the system.

Message-Passing Platforms

43

- These platforms comprise of a set of processors and their own (exclusive) memory.
- Instances of such a view come naturally from clustered workstations and non-shared-address-space multicomputers.
- These platforms are programmed using (variants of) send and receive primitives. • Libraries such as MPI and PVM provide such primitives

Message Passing vs. Shared Address Space Platforms

44

- Message passing requires little hardware support, other than a network.
- Shared address space platforms can easily emulate message passing. The reverse is more difficult to do (in an efficient manner).

M1:

45

- Motivating Parallelism, Scope of Parallel Computing,
- Introduction to HPC:
- Parallel Programming Platforms
 - Implicit Parallelism: Trends in Microprocessor Architectures
 - Limitations of Memory System Performance
- Dichotomy of Parallel Computing Platforms
 - Control Structure of Parallel Platforms
 - Communication Model of Parallel Platforms
- Physical Organization of Parallel Platforms
 - Architecture of an Ideal Parallel Computer
 - Interconnection Networks for Parallel Computers

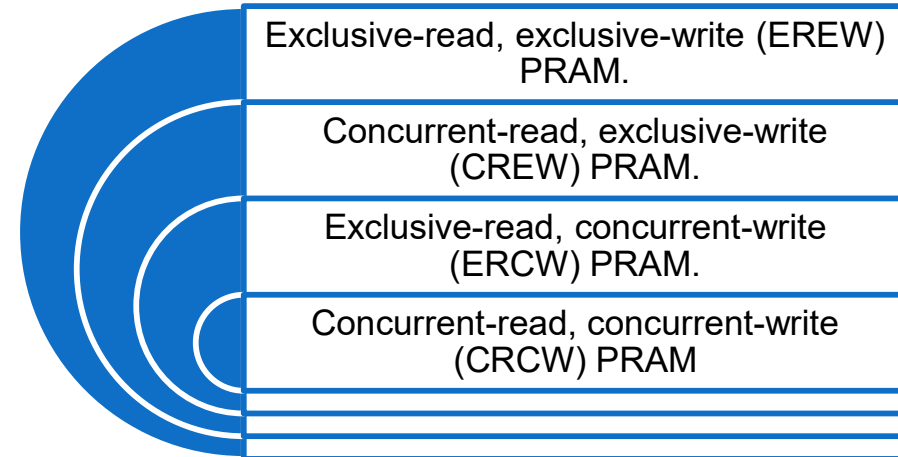
Physical Organization of Parallel Platforms

46

Architecture of an Ideal Parallel Computer

- parallel random access machine (PRAM)
- PRAMs consist of p processors and a global memory of unbounded size that is uniformly accessible to all processors.
- Processors share a common clock but may execute different instructions in each cycle
- Simultaneous memory accesses are handled, PRAMs can be divided into four subclasses

PRAMs subclasses



concurrent read access does not create any semantic discrepancies

concurrent write access to a memory location requires arbitration

<https://bheemhh.github.io/#/HPC26>

Architecture of an Ideal Parallel Computer

47

What does concurrent write mean ?

- The most frequently used protocols are as follows:
 - **Common**, in which the concurrent write is allowed if all the values that the processors are attempting to write are identical.
 - **Arbitrary**, in which an arbitrary processor is allowed to proceed with the write operation and the rest fail.
 - **Priority**, in which all processors are organized into a predefined prioritized list, and the processor with the highest priority succeeds and the rest fail.
 - **Sum**, in which the sum of all the quantities is written (the sum-based write conflict resolution model can be extended to any associative operator defined on the quantities being written)

Architectural Complexity of the Ideal Model

48

- Processors and memories are connected via **switches**.
- System of p processors and m words, the switch complexity is $O(mp)$.
- For a reasonable memory size, constructing a switching network of this complexity is very expensive.
- Thus, PRAM models of computation are impossible to realize in practice
- Solution :
 - **Interconnection Networks** for Parallel Computers
 - A blackbox view of an interconnection network consists of **n inputs and m outputs**

M1:

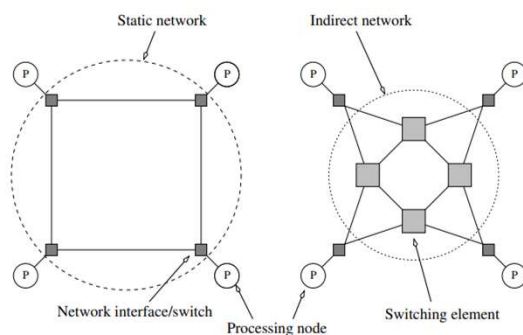
49

- Motivating Parallelism, Scope of Parallel Computing,
- Introduction to HPC:
- Parallel Programming Platforms
 - Implicit Parallelism: Trends in Microprocessor Architectures
 - Limitations of Memory System Performance
- Dichotomy of Parallel Computing Platforms
 - Control Structure of Parallel Platforms
 - Communication Model of Parallel Platforms
- Physical Organization of Parallel Platforms
 - Architecture of an Ideal Parallel Computer
 - Interconnection Networks for Parallel Computers

Interconnection Networks for Parallel Computers

50

- Interconnects are made of switches and link
- Interconnects are classified as static or dynamic.
- **Static networks** consist of point-to-point communication links among processing nodes and are also referred to as direct networks.
- **Dynamic networks** are built using switches and communication links. Dynamic networks are also referred to as indirect networks

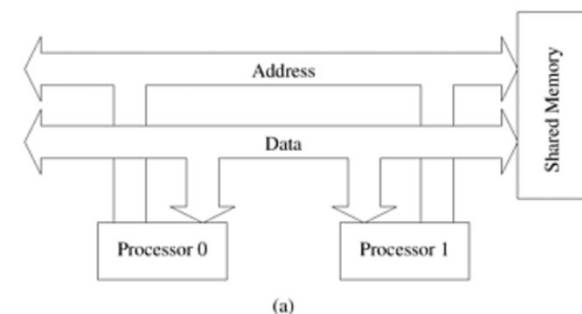


Classification of interconnection networks: (a) a static network; and (b) a dynamic network.

Interconnection Networks Components

51

- **Switches :**
 - a fixed number of inputs to outputs; Degree of the switch : total number of ports
- **Network Interfaces:**
 - Processors talk to the network via a network interface.
 - connectivity between the nodes and the network
- **Network Topologies:**
 - Bus-Based Networks :
 - Buses are also ideal for broadcasting information among nodes
 - transmission medium is shared, there is little overhead associated with broadcast compared to point-to-point message transfer
 - Reducing shared-bus bandwidth using caches



<https://bheemhh.github.io/#/HPC26>